
# HTTPS, 왜 안전할까?

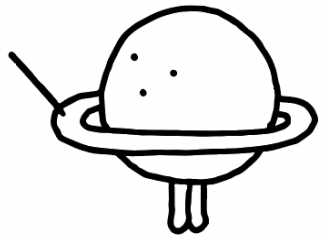
우아한테크코스 BE 7기 메이

# 이 발표의 타겟층

HTTPS 통신하는 서비스를 운영해본 경험이 있는 사람

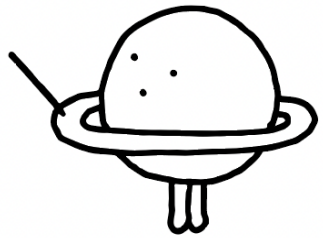
HTTPS 환경 구축할때 AI가 알려준대로 따라가기만 한 사람

인증서가 어떤 용도로 사용되는지 궁금한 사람

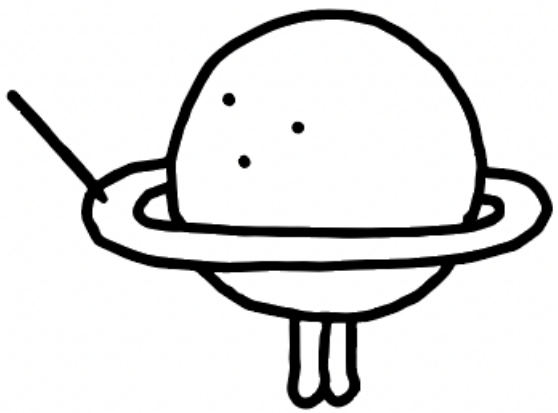


# 목차

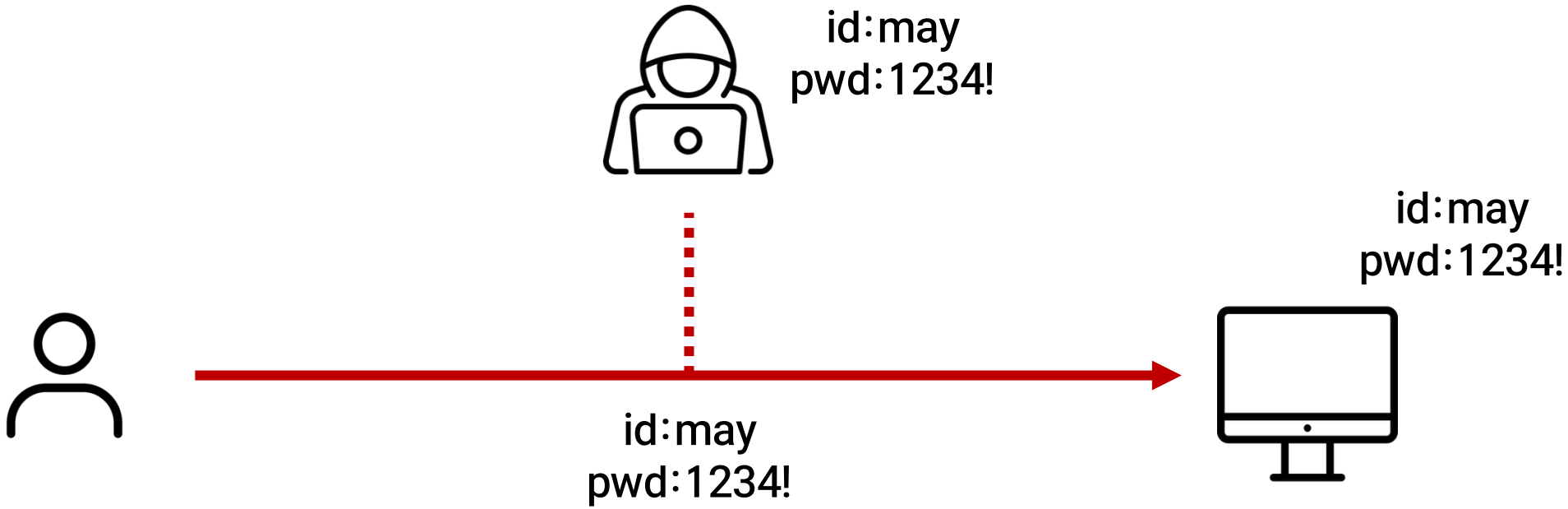




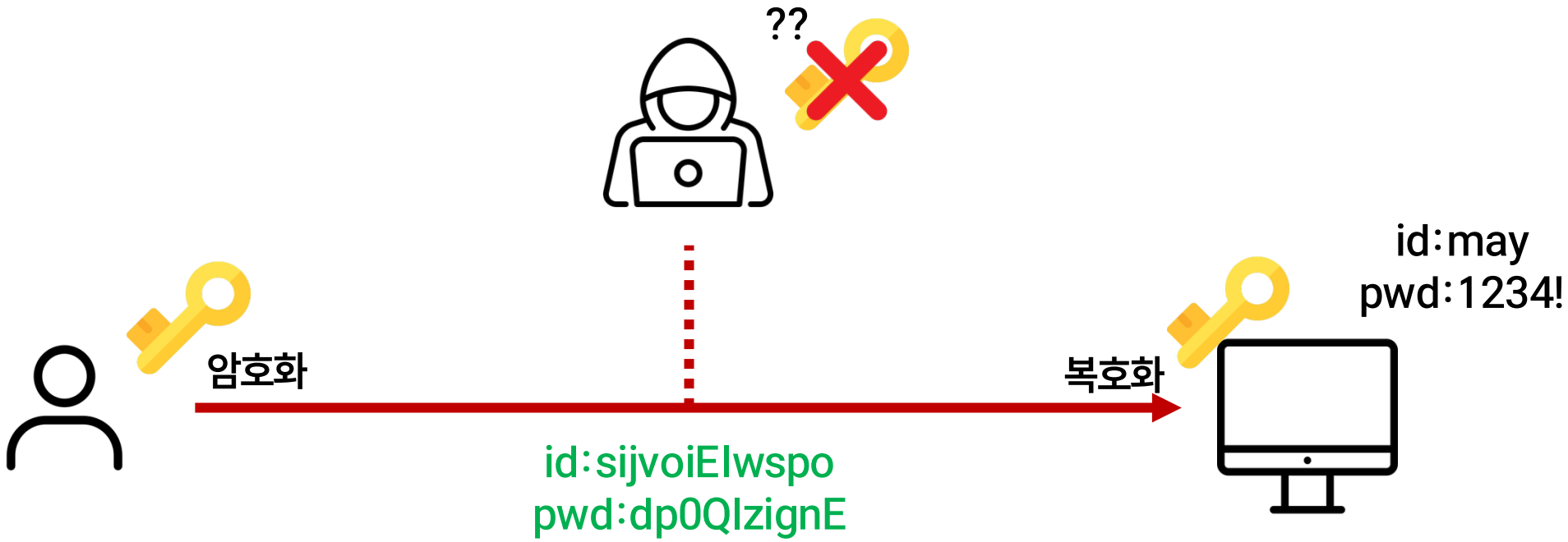
# 서론



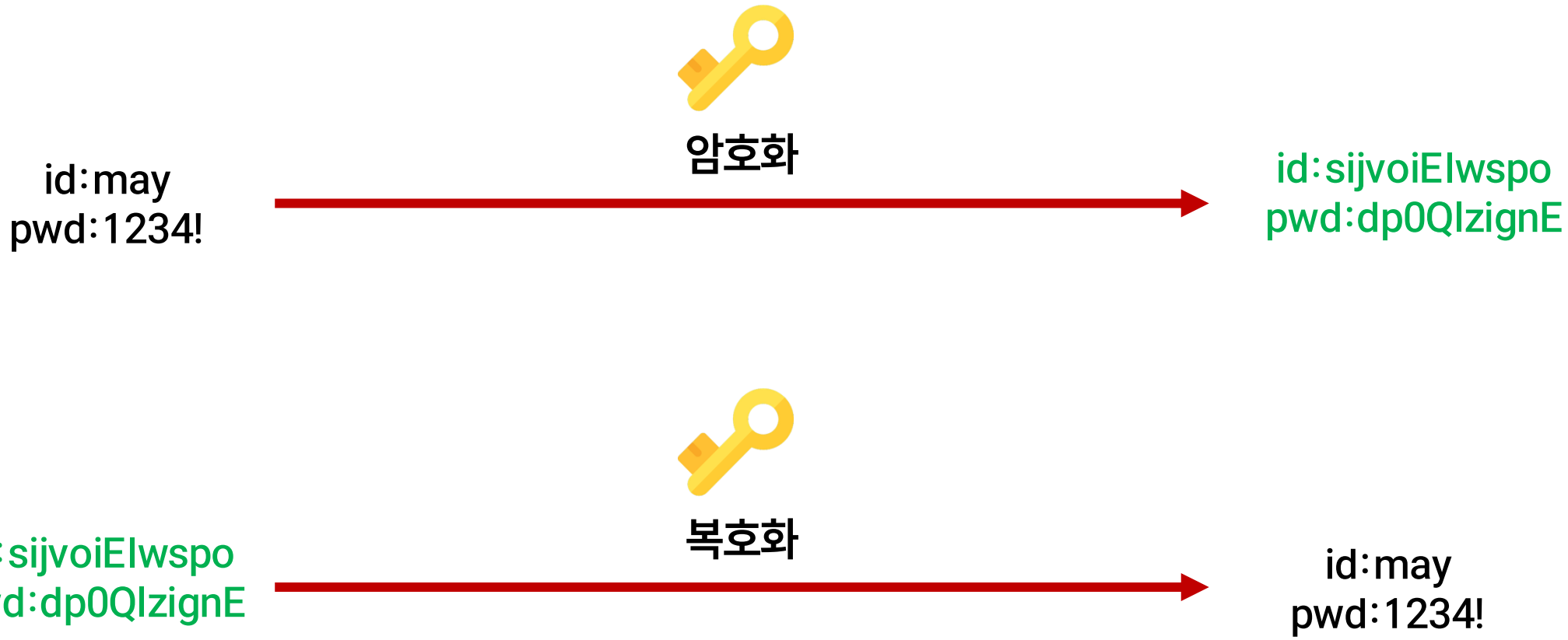
# HTTP



# HTTPS(HTTP over TLS)



# 대칭키 암호화(Symmetric Key Encryption)




**암, 복호화에 동일한 key(대칭키) 사용**

# 대칭키 암호화 알고리즘 – AES(Advanced Encryption Standard)

**DES 사용**
1977년 미국 정부 표준

↓

**취약점 발견**
56비트 키, 무차별 대입 가능

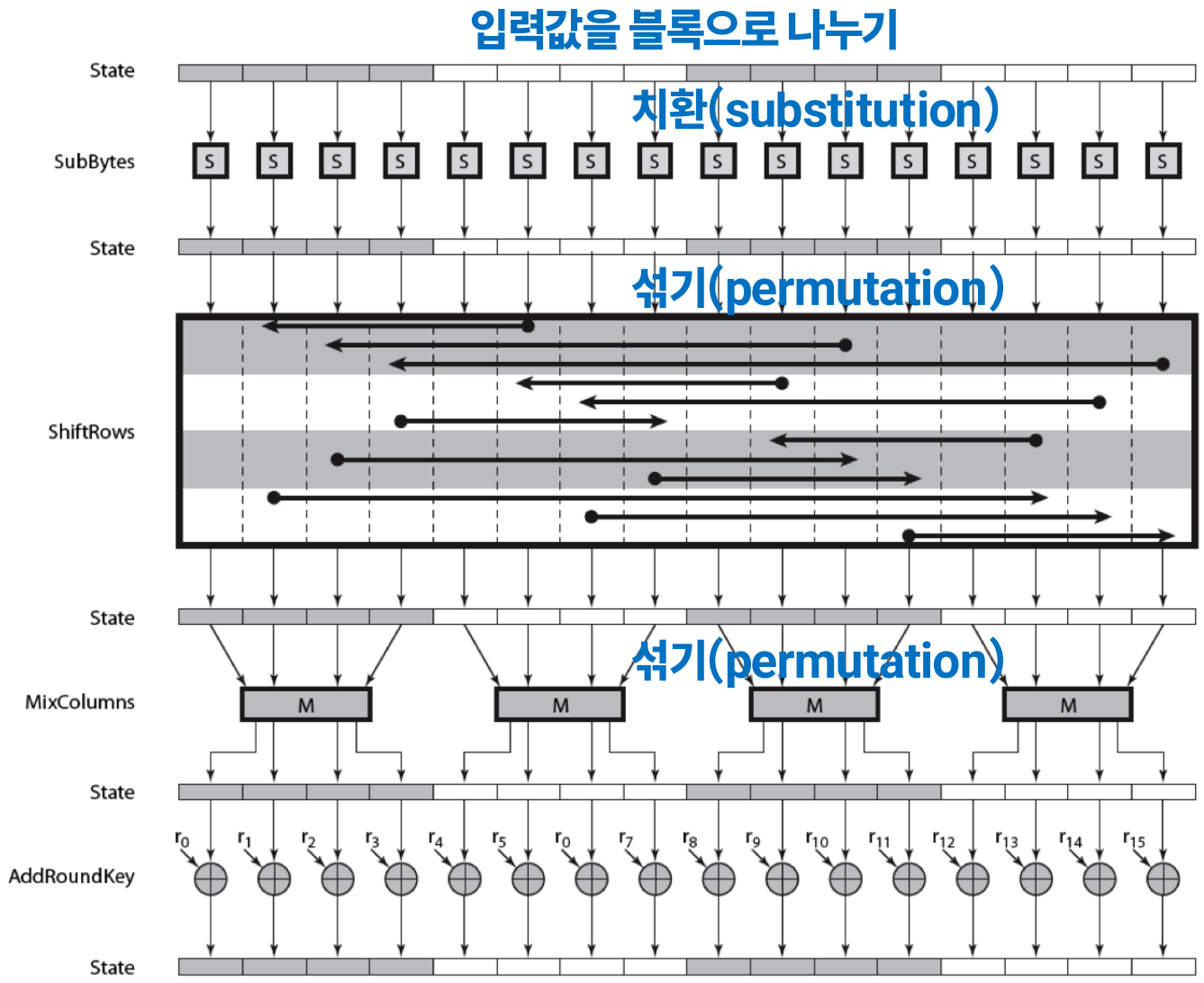
↓

**AES 공모전 개최**
1997년 NIST 주관

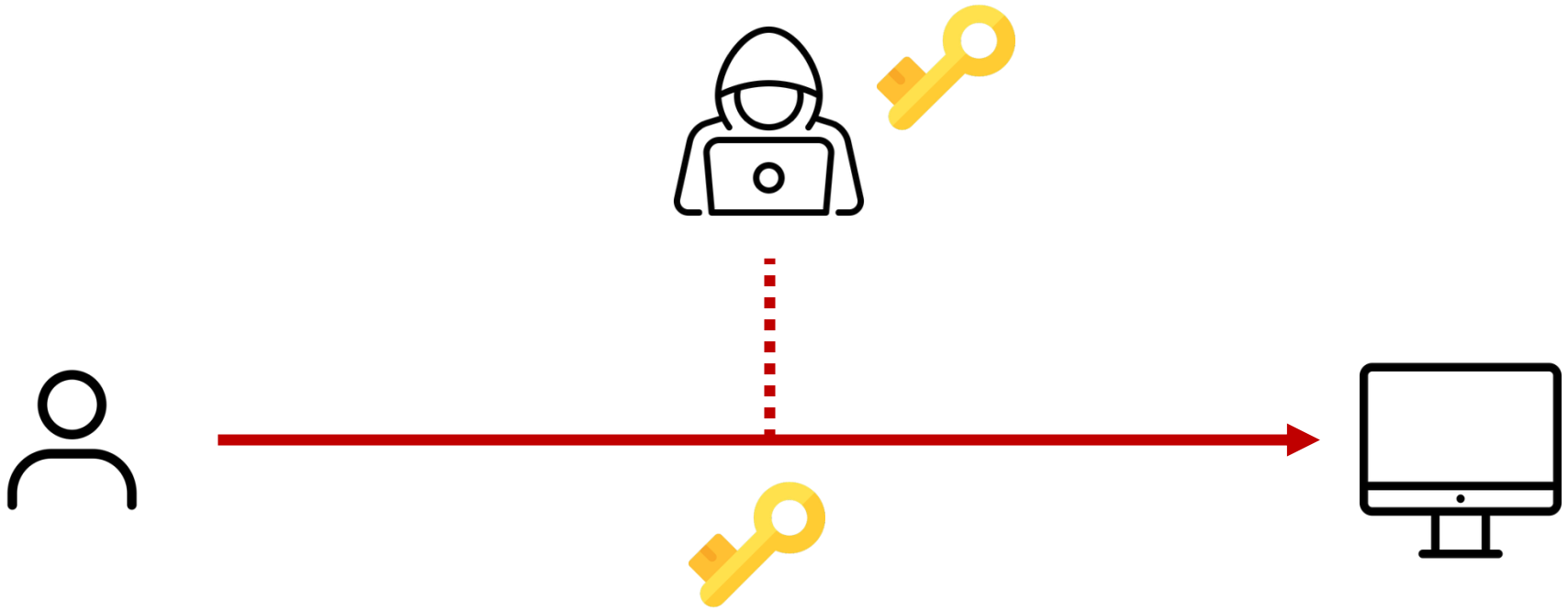
↓

**AES 선정**
2001년 Rijndael 채택

## 공모전에서 선정된 표준

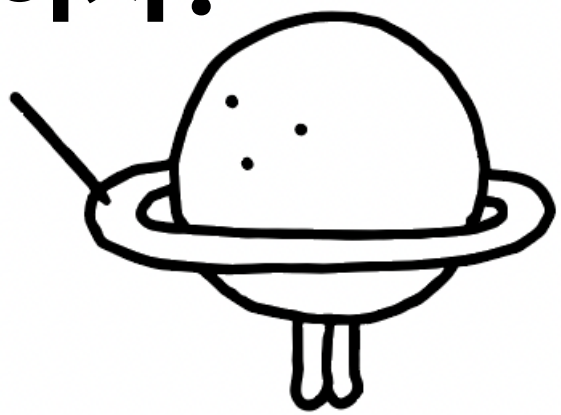



## AES 암호화 과정

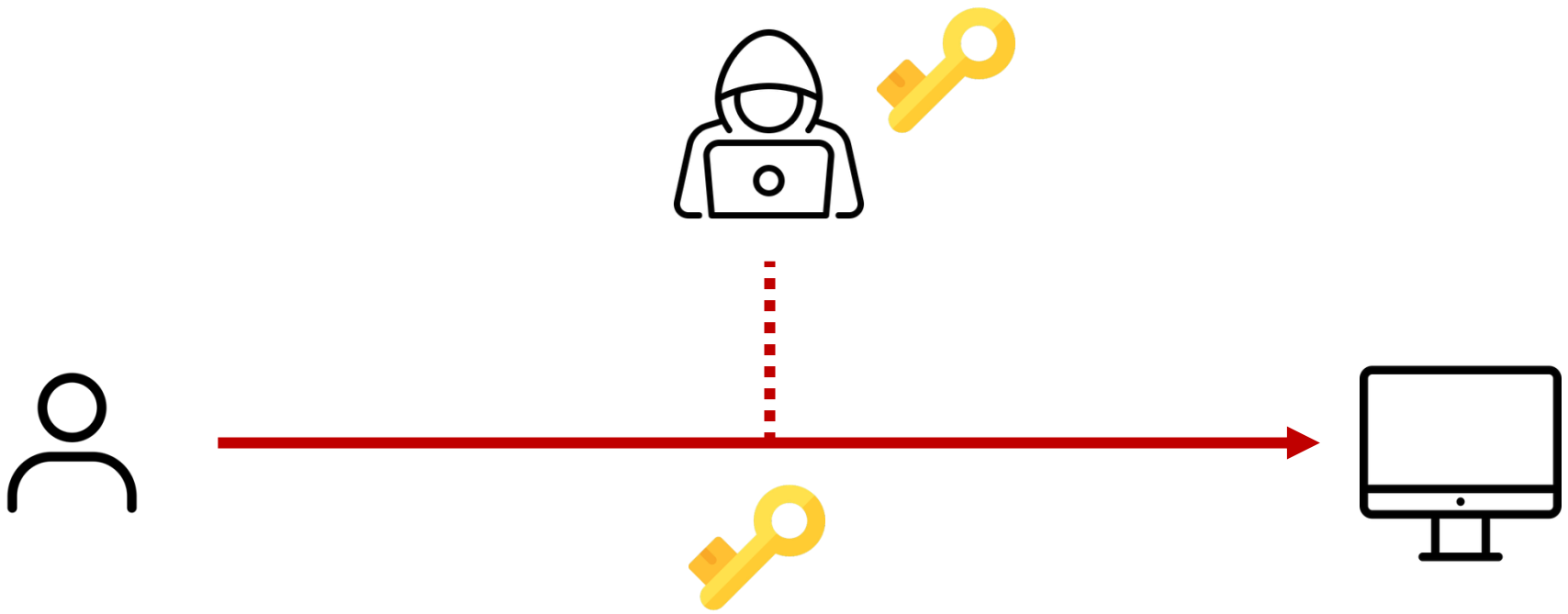



## 암호화 키를 어떻게 안전하게 교환할 수 있을까?

# Key 교환 어떻게 하지?



# Key exchange



# Diffie-Hellman Key Exchange

돔푸

메이

암호화키 교환을 위한
비대칭키

Private key: d
Public key: $g^d$

Private key: m
Public key: $g^m$

$g^d$

$g^m$

돔푸의 private key

암호화에 사용할
대칭키

대칭키 생성: $(g^m)^d$

메이의 public key

대칭키 생성: $(g^d)^m$

# Man In The Middle Attack(MITM)

돔푸

Private key: d
Public key: $g^d$

젠슨

Private key: g
Public key: $g^g$

메이

Private key: m
Public key: $g^m$

$g^d$ →

← $g^g$

$g^g$ →

← $g^m$

대칭키 생성: $(g^g)^d$

대칭키 생성: $(g^d)^g$ $(g^m)^g$

대칭키 생성: $(g^g)^m$

# Man In The Middle Attack(MITM) 방어 아이디어

돔푸

Private key: d
Public key: $g^d$

젠슨

Private key: g
Public key: $g^g$

메이

Private key: m
Public key: $g^m$

$g^d$ →

← $g^g$

$g^g$ →

← $g^m$

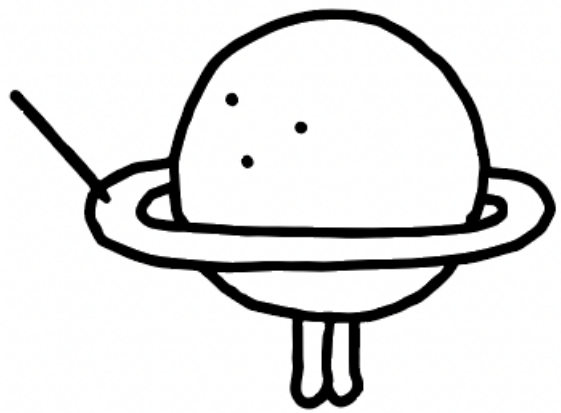
대칭키 생성: $(g^g)^d$

대칭키 생성: $(g^d)^g$
$(g^m)^g$

대칭키 생성: $(g^g)^m$

여기서 받은 public key가 메이의 public key임을
인증(Authenticate)할 수 없을까?

# 인증서



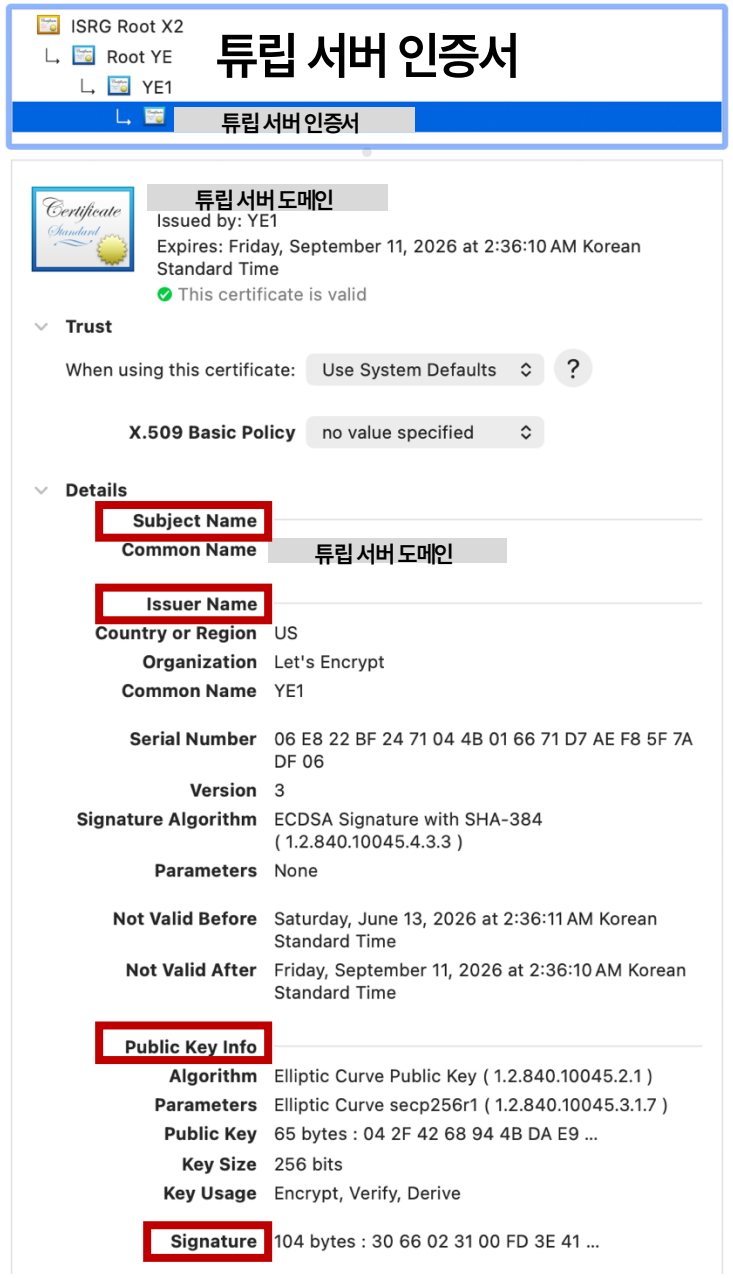
# 인증서(Certificate) 발급

```
sudo certbot certificates

Saving debug log to /var/log/letsencrypt/letsencrypt.log

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Found the following certs:
  Certificate Name:
    Serial Number: 6e822bf2471044b016671d7aef85f7adf06
    Key Type: ECDSA
    Domains:
    Expiry Date: 2026-09-10 17:36:10+00:00 (VALID: 79 days)
    Certificate Path: /etc/letsencrypt/live/turip.p-e.kr/fullchain.pem
    Private Key Path: /etc/letsencrypt/live/turip.p-e.kr/privkey.pem
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

서버의 인증서 정보
(public key 포함)

서버의 개인키 정보

# 인증서(Certificate) 내용




subject: 인증서의 주인

issuer: 인증서를 발급해준 기관

public key: 서버의 공개키 정보

signature: 인증서 위조 방지를 위한 서명값

나 정말 튜립이야..

# CA(Certificate Authority)

CA(issuer)

이 기관을 신뢰해야

발급

이 인증서도 신뢰 가능

# CA(Certificate Authority)

ISRG Root X2

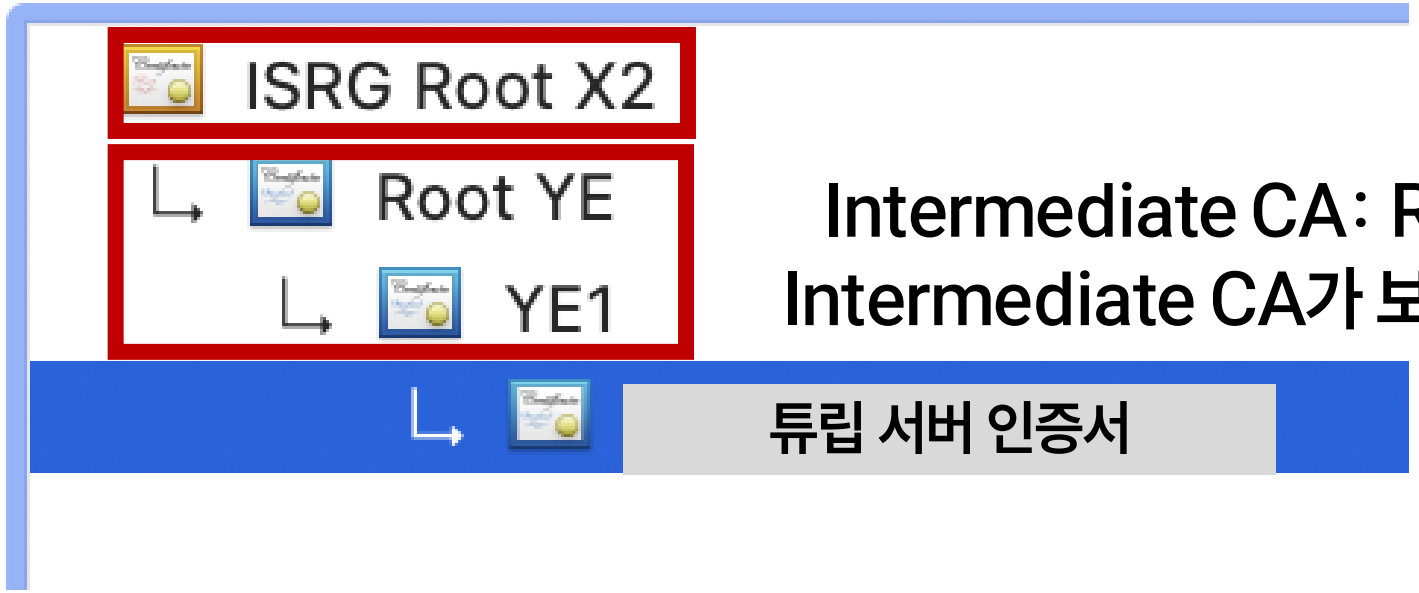
Client의
Trust Store

ISRG Root X2
DigiCert Root
Apple Root CA
...

Client의 가 신뢰하는 CA라서
인증서 신뢰 가능

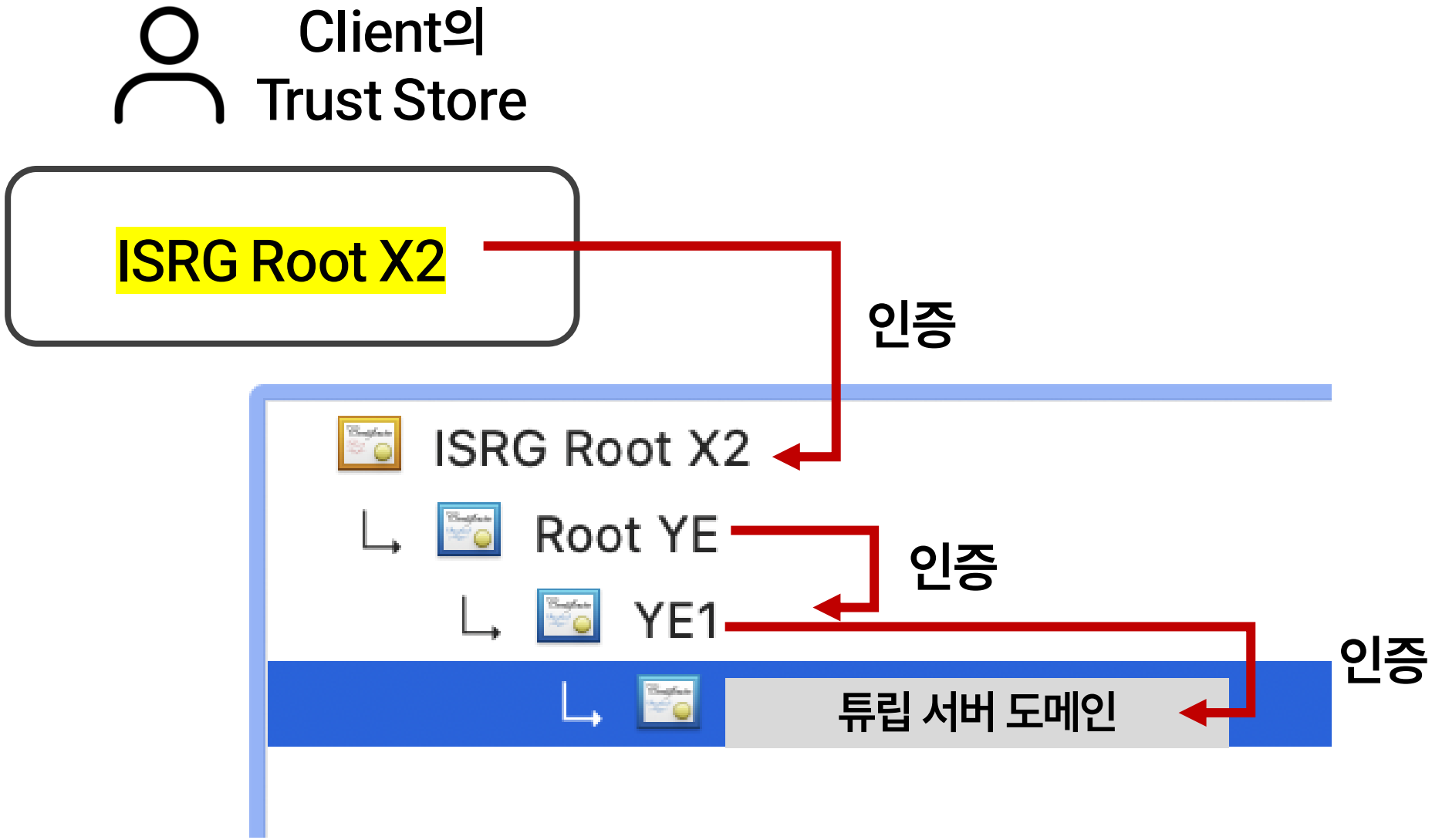
# Certificate Chain

Root CA: 공식적으로 인정받은 CA

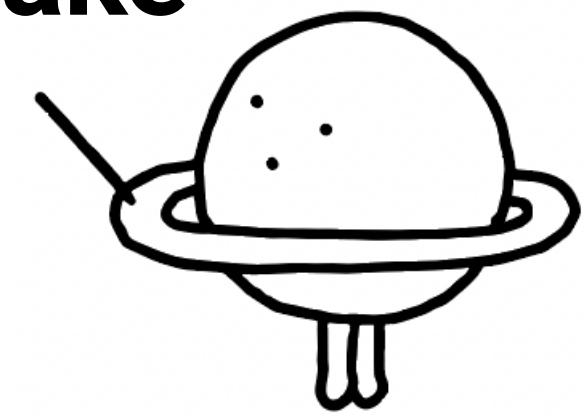



Intermediate CA: Root CA,
Intermediate CA가 보증하는 CA

# Certificate Chain



# TLS Handshake

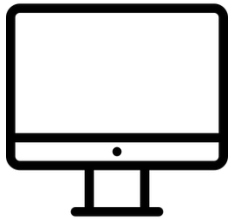


# 중간 정리

id:may
pwd:1234!

암호화

대칭키
(암호화 키)



id:sijvoiElwspo
pwd:dp0QlzignE

복호화

대칭키
(암호화 키)

id:sijvoiElwspo
pwd:dp0QlzignE

id:may
pwd:1234!

소통하는 개체끼리 암/복호화 하기 위한 대칭키(암호화 키)가 필요하다

# 중간 정리

Private key: c
Public key: $g^c$

Private key: s
Public key: $g^s$

$g^c$

$g^s$

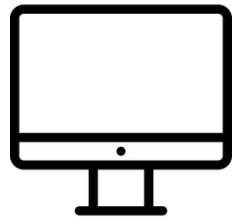
대칭키 생성: $(g^s)^c$

대칭키 생성: $(g^c)^s$

대칭키(암호화 키)는 비대칭 키를 사용해서 계산한다

# 중간 정리

Private key: c
Public key: $g^c$



Private key: s
Public key: $g^s$

$g^c$

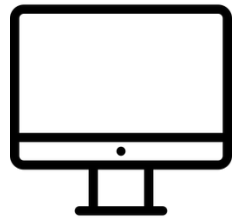

진짜 serve의 public key가 맞네
대칭키 생성: $(g^s)^c$

대칭키 생성: $(g^c)^s$

어떤 개체의 public key가 진짜 그 개체의 것임을 인증하기 위해 certificate을 사용한다
(MITM attack 방지)

# 중간 정리

Private key: c
Public key: $g^c$



Private key: s
Public key: $g^s$

$g^c$

$g^s$

진짜 serve의 public key가 맞네
대칭키 생성: $(g^s)^c$

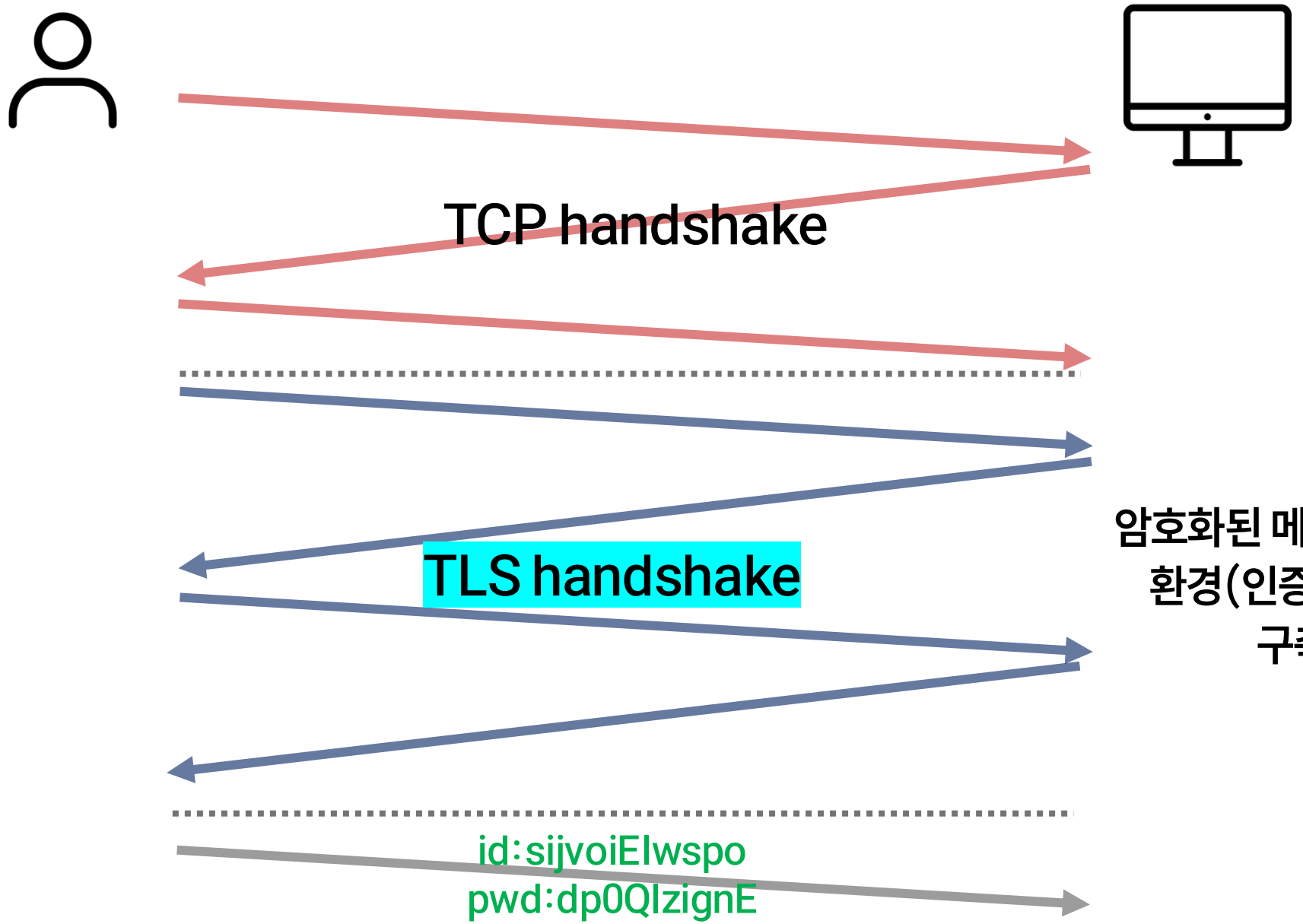
대칭키 생성: $(g^c)^s$

id:may
pwd:1234!

id:sijvoiElwspo
pwd:dp0QlzignE

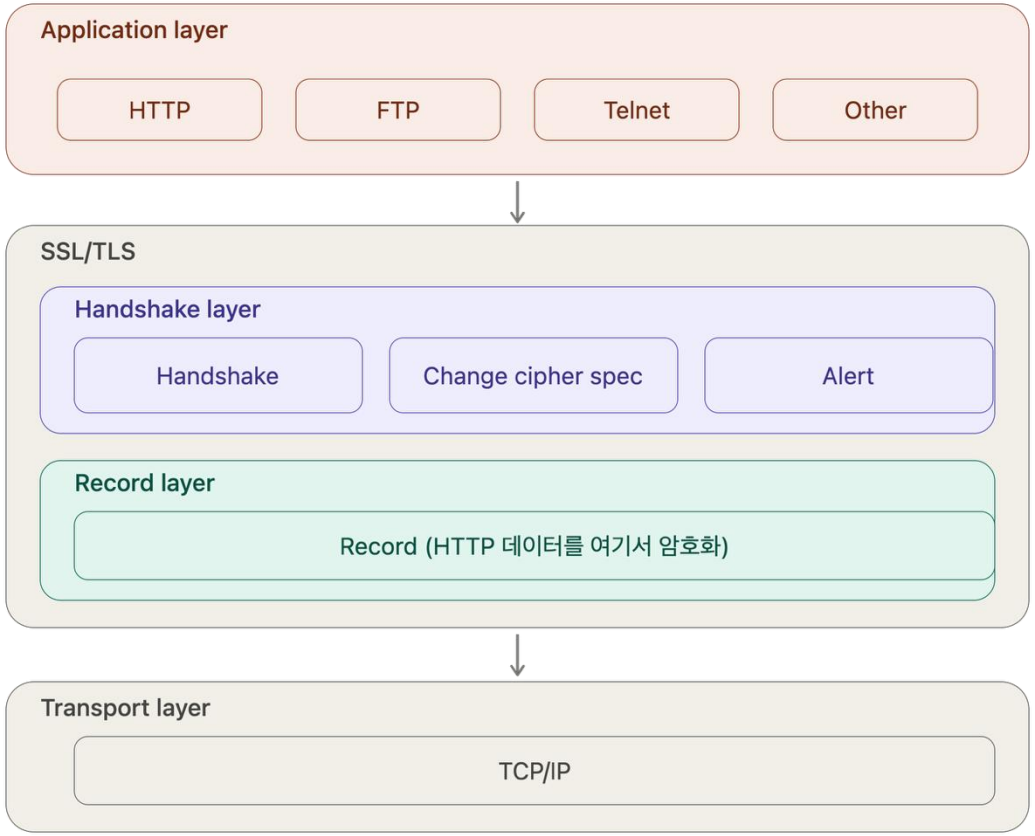
암호화 통신 가능

# HTTPS 통신을 위해 필요한 과정




암호화된 메시지를 주고 받을 수 있는
환경(인증서 검증, 키 생성 …)을
구축하기 위한 과정

# TLS(Transport Layer Security) Protocol

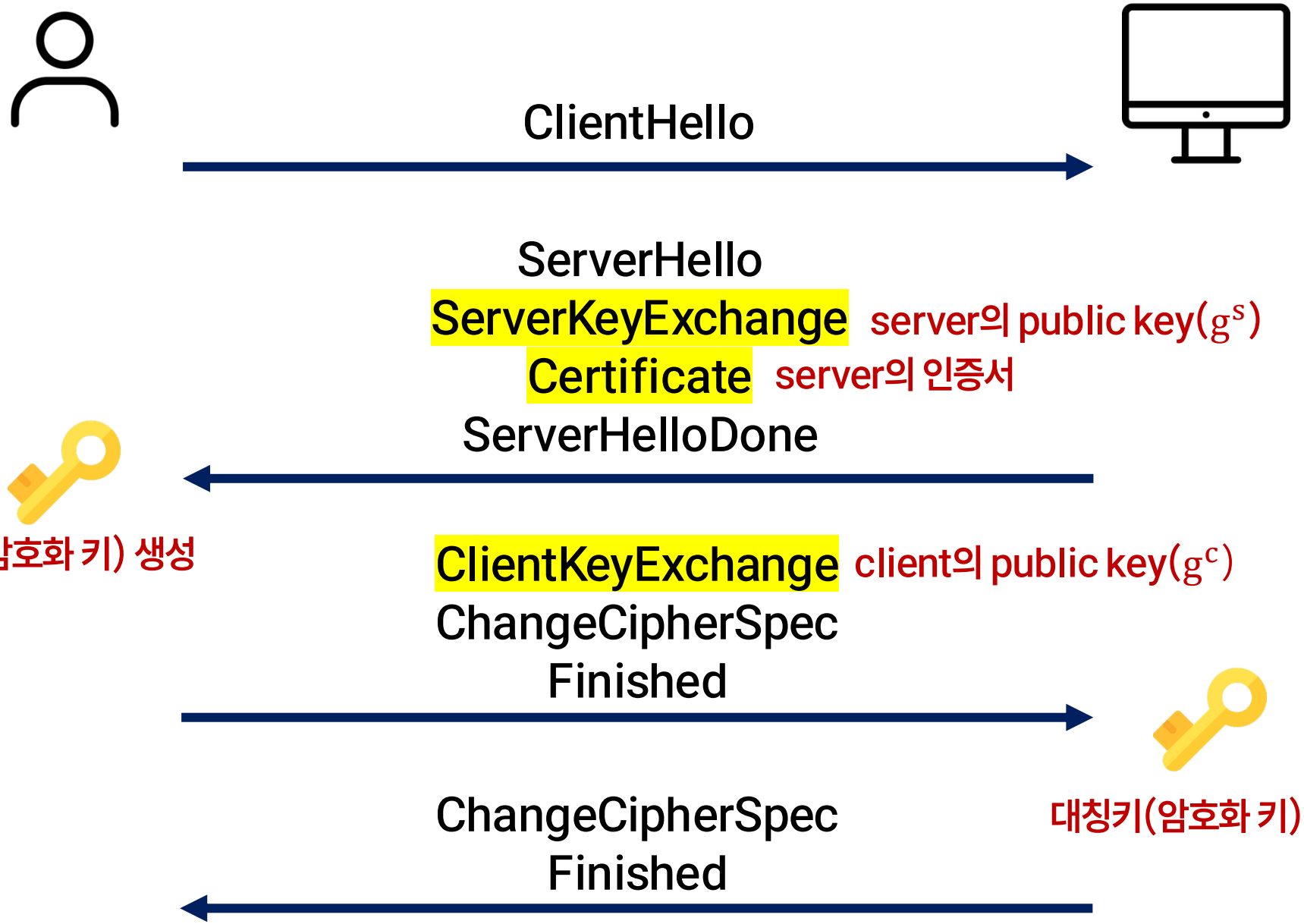



Transport Layer 위에서
Application Layer의 데이터 암호화

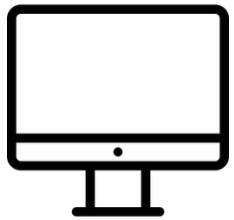
TLS → 버전 1.2 ~ 1.3 사용

인증/암호화/무결성 제공

# TLS 1.2 Handshaking



# TLS 1.2 Handshaking



ClientHello

ServerHello
ServerKeyExchange
Certificate
ServerHelloDone

암호화 메시지를 보내기까지
2-RTT 필요

ClientKeyExchange
ChangeCipherSpec
Finished

ChangeCipherSpec
Finished

# TLS 1.3 Handshaking(1-RTT)

client의 public key($g^c$)

ClientHello(key share)

server의 public key($g^s$)

ServerHello(key share)

Certificate server의 인증서

Finished

대칭키(암호화 키) 생성

대칭키(암호화 키) 생성

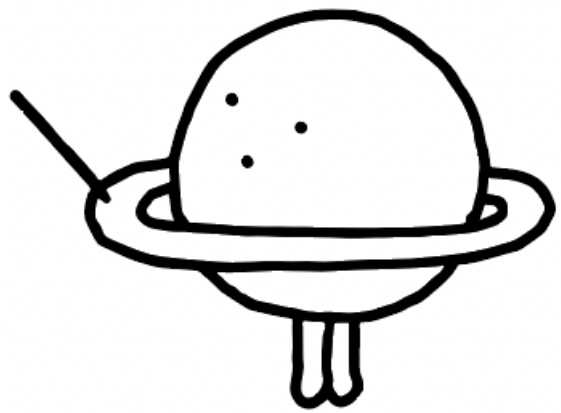
Finished

+ 암호화 메시지

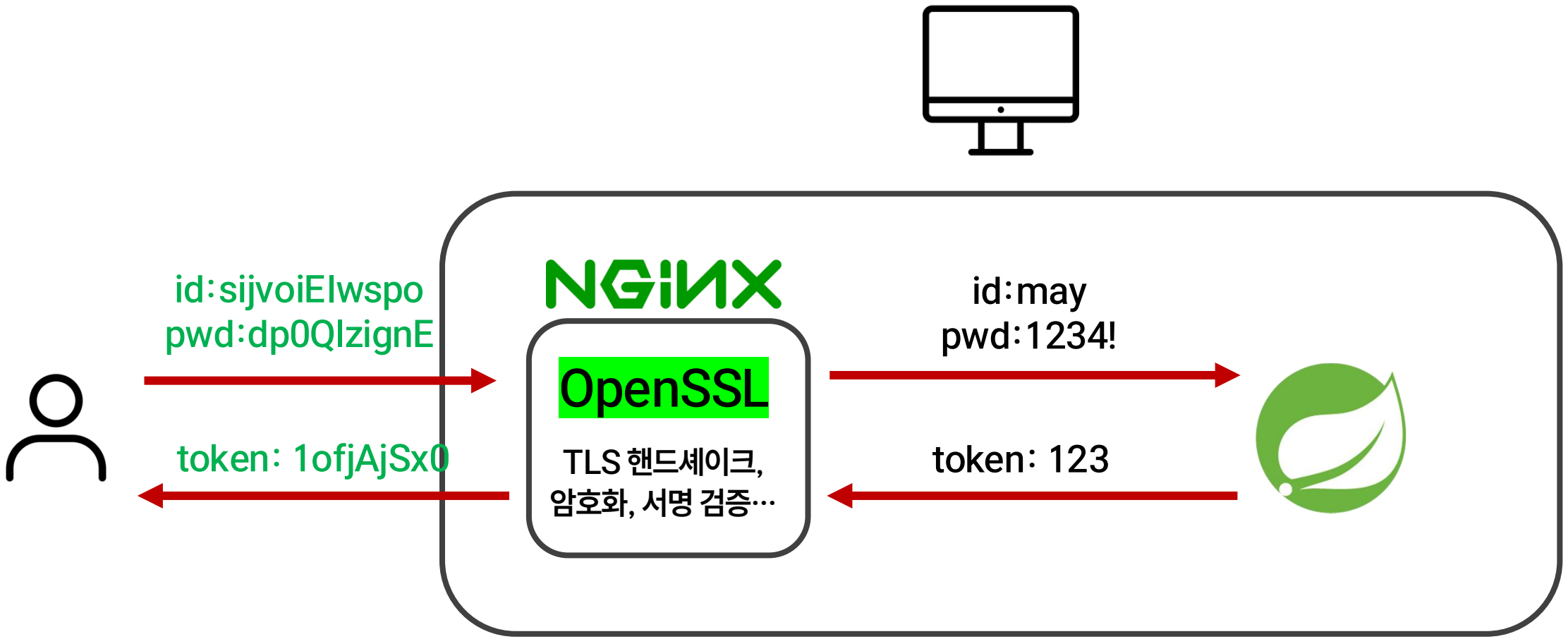
암호화 메시지를 보내기까지

1-RTT 필요

# 튜립 서버의 HTTPS 설정



# 튜립의 HTTPS 설정


id:sijvoiElwspo
pwd:dp0QlzignE
NGINX
OpenSSL
TLS 핸드셰이크,
암호화, 서명 검증…
id:may
pwd:1234!
token: 123
token: 1ofjAjSx0

# 튜립의 HTTPS 설정

## nginx -> openSSL 요청 설정 파일(튜립의 암호화 통신 관련 설정)

```
cat /etc/letsencrypt/options-ssl-nginx.conf

# This file contains important security parameters. If you modify this file
# manually, Certbot will be unable to automatically provide future security
# updates. Instead, Certbot will print and log an error message with a path to
# the up-to-date file that you will need to refer to when manually updating
# this file. Contents are based on https://ssl-config.mozilla.org

ssl_session_cache shared:le_nginx_SSL:10m;
ssl_session_timeout 1440m;
ssl_session_tickets off;

ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers off;

ssl_ciphers "ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-E
CDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:DHE-RSA-AES128-GCM-SHA256:DHE-RSA-AES256-GCM-SHA384":
```

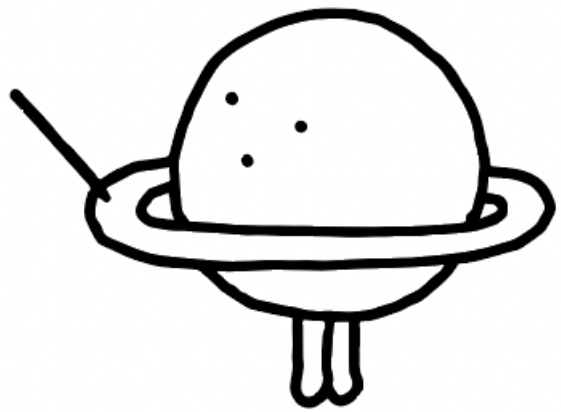
TLS 1.2, 1.3만 허용

cipher suite(허용하는 알고리즘 목록)

**ECDHE - ECDSA - AES128-GCM - SHA256**

- ECDHE: 키교환 알고리즘
- ECDSA: 인증서 서명 알고리즘
- AES128-GCM: 대칭키 암호화 알고리즘
- SHA256: 해시 알고리즘

# OpenSSL 취약점, 신경써야 할 것



# 과거 OpenSSL의 취약점 – HeartBleed(2014.04)

## Heartbleed: Heartbeat 요청을 통해 정보 빼오는 공격

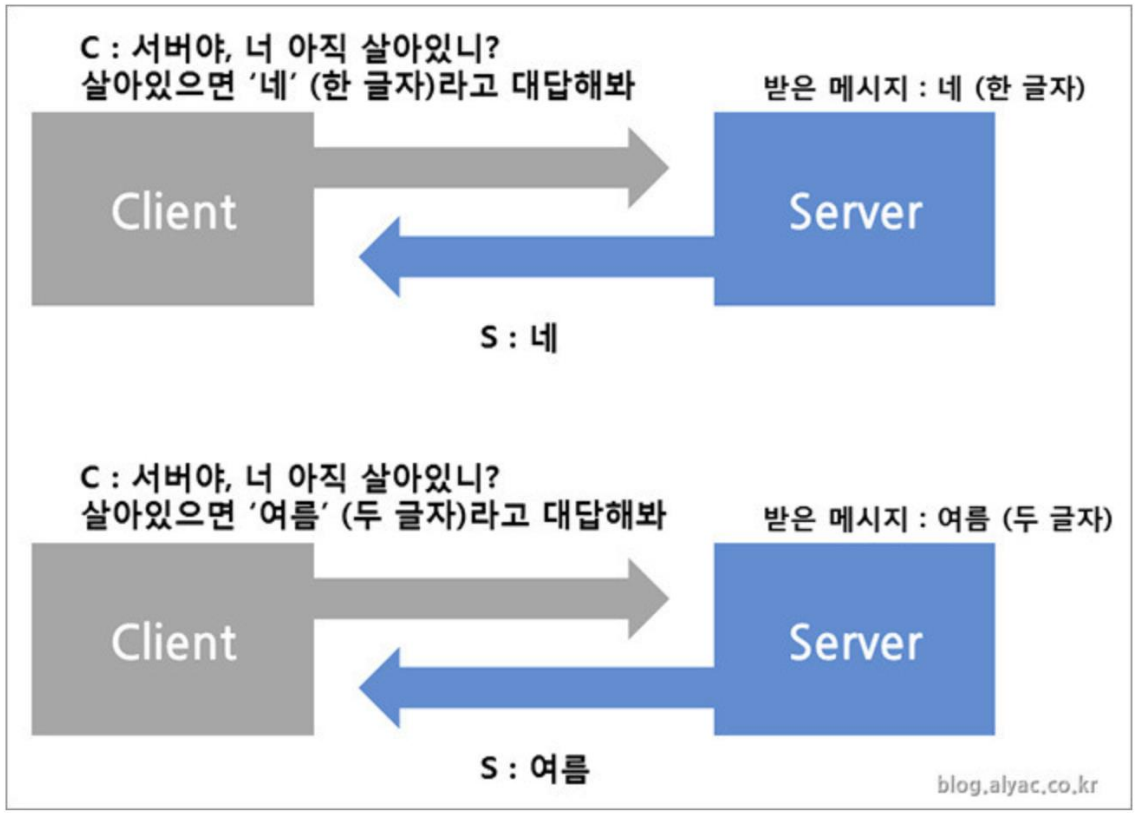



그림1. 정상적인 요청과 정상적인 반환값

이미지 출처: https://blog.alyac.co.kr/76

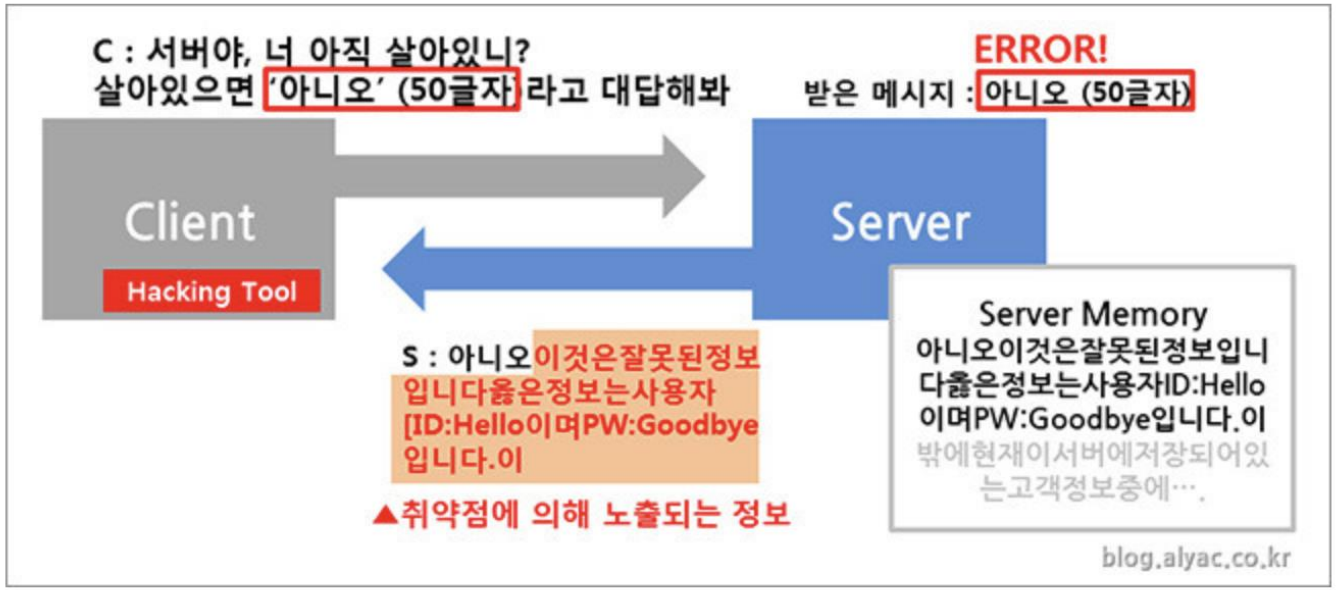



그림2. payload의 길이를 조작하여 정보 유출

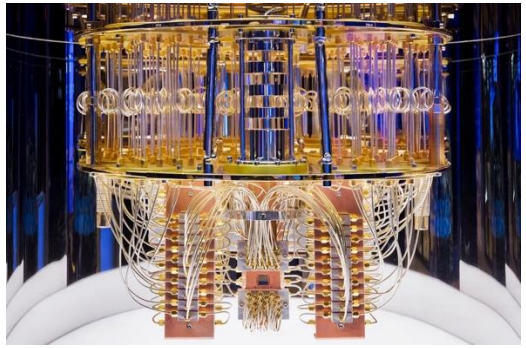
## 병원, 금융 등 약 17%의 서버가 영향을 받음

# HNDL(Harvest Now Decrypt Later)

현재 안전한 ECDHE 같은 키 교환 방식이 미래의 양자컴퓨터에 의해 깨질 수 있다

## 2026: 데이터 수집해두기

서버 public key($g^c$ ),
클라이언트 public key($g^s$),
통신 데이터(pwd:dp0QlzignE)
...



## 20xx: 양자컴퓨터 개발! 수집해둔 데이터 복호화

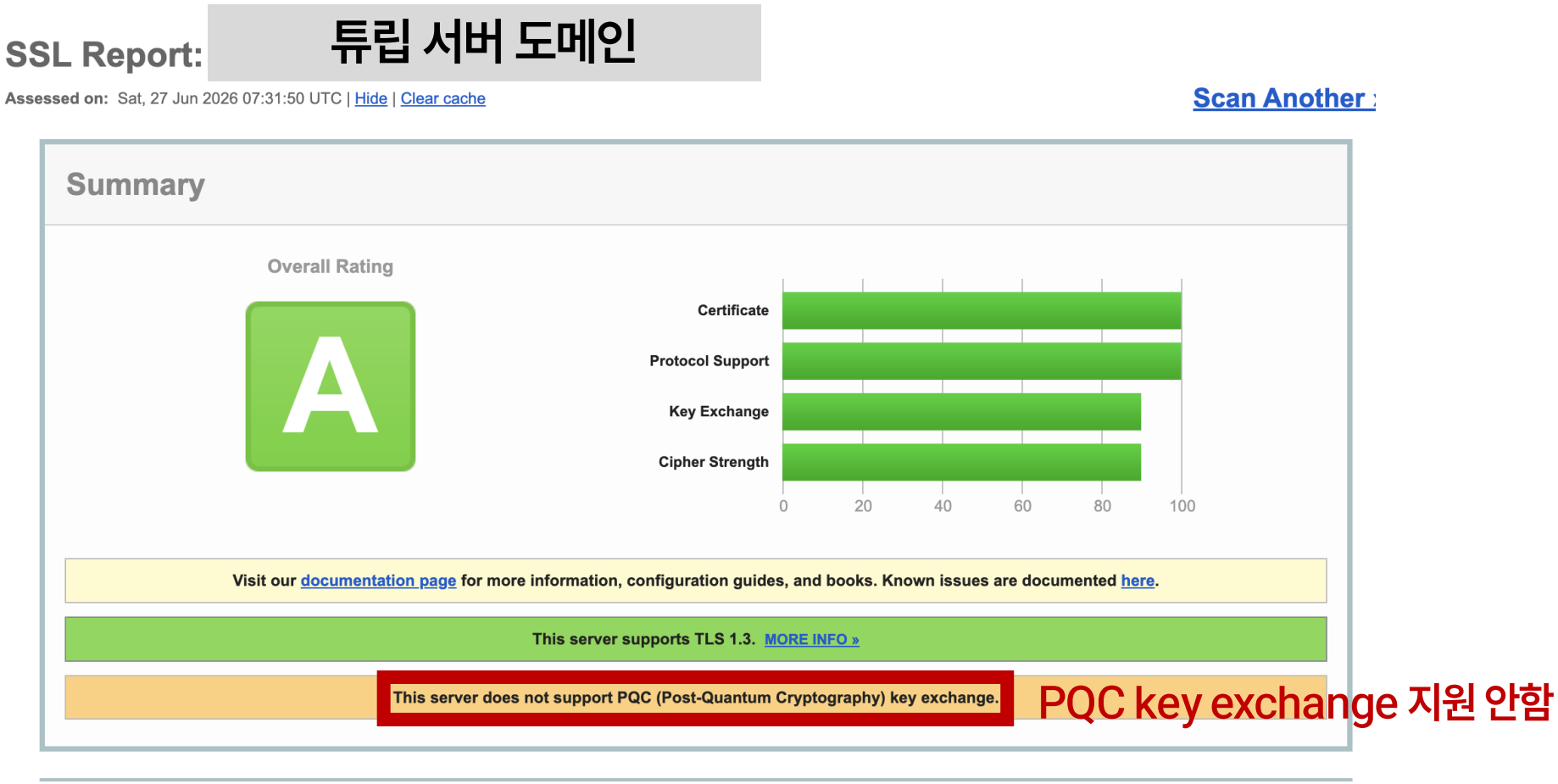
$g^c$, $g^s$ 기반 대칭키($g^{sc}$ ) 계산 성공!
통신 데이터(pwd:1234) 복호화 성공!

# PQC(Post Quantum Cryptography)

양자컴퓨터로 깨기 어려운 키교환 알고리즘

# OpenSSL에 의존하는 서버에서 신경써야 할 점

- CVE(취약점) 추적 - 현실적으로 힘들듯
- 최신 버전 사용을 위해 자동 업데이트(unattended-upgrades)
- PQC 적용하기 - 우리 서버가 양자컴퓨터 나올때까지 유효할거같고 누군가 지금 우리 서버의 데이터를 미리 수집할 것 같다면



# Q&A